



← VLSIDA/OpenRAM

给出完整的代码地图

Codemap

Open with Windsurf

OpenRAM SRAM编译器完整代码地图

OpenRAM是一个开源SRAM编译器，本地图展示从用户配置到最终输出生成的完整流程。关键入口点包括主编译器脚本、SRAM实例创建和字符化分析流程。

1 SRAM编译器主流程

展示用户运行SRAM编译器时的完整执行路径，从配置解析到文件输出生成. [See guide](#)

AI generated guide

Motivation

在ASIC设计中，**内存编译器**是必不可少的工具。设计师需要根据不同的应用需求生成各种规格的SRAM（静态随机存取存储器），从小的配置寄存器到大的缓存存储器。手动设计每种规格的SRAM既耗时又容易出错。OpenRAM解决了这个问题，它是一个**开源的自动化内存编译器**，能够根据用户指定的参数（如字长、字数、端口数、工艺节点等）自动生成完整的SRAM设计，包括电路网表、物理布局、时序模型等所有必要文件。

Details

OpenRAM的工作流程从[1a]系统初始化开始，解析用户配置文件并加载相应的工艺参数。系统支持多种工艺节点，每个工艺都有专门的设计规则和器件模型。

核心流程在[1b]创建SRAM实例时启动。系统会根据配置参数**自动计算最优的内存组织结构**，包括确定行列数量、银行划分等。这个计算过程考虑了位单元的物理尺寸和电气特性，以实现最佳的面积和性能平衡。

[1c]保存输出文件阶段生成完整的设计交付物：

- **SPICE网表**用于电路仿真验证
- **GDSII布局**用于芯片制造
- **LEF模型**用于数字布局布线工具
- **Liberty时序库**用于静态时序分析
- **Verilog行为模型**用于系统级仿真

sram_compiler.py 主脚本入口

整个编译过程是**完全自动化**的，用户只需要提供配置文件，系统就会完成从架构设计到物理实现再到字符化分析完整流程。这大大提高了内存设计的效率和质量。

```
1a 初始化OpenRAM系统
    openram.init_openram(config_file=args[0])
    — 初始化技术参数
    — 设置全局选项
    — 加载工艺库文件
— 创建SRAM实例
    1b 创建SRAM实例
        from openram import sram s = sram()
        — 计算内存组织结构
        — 生成电路网表
```

sram_compiler.py:42

sram_compiler.py:72

Ask a follow-up question



- GDS布局输出
- LEF物理模型
- Liberty时序库
- Verilog行为模型

清理临时文件和退出

2 SRAM架构创建与组织

展示SRAM内存架构的动态生成过程，包括配置计算和银行创建. [See guide](#)

🔗 AI generated guide

Motivation

OpenRAM解决了**自动生成定制SRAM存储器**的复杂问题。在芯片设计中，手工设计SRAM需要耗费数周时间，涉及复杂的电路设计、版图布局和时序分析。OpenRAM通过自动化流程，让用户只需提供简单的配置参数（如字长、字数、工艺节点），就能在几分钟内生成生产级的SRAM设计，包括SPICE网表、GDS版图、LEF物理模型和Liberty时序库文件。

Details

SRAM架构创建过程从[1a]系统初始化开始，首先解析用户配置文件并设置技术参数。核心创建流程在[2b]处触发，系统会创建一个sram_1bank实例作为主要架构。配置参数通过[2c]传播到各个模块，确保所有组件使用一致的设计规格。最关键的是[2d]的compute_sizes()算法，它根据位单元的物理尺寸和目标容量，智能计算最优的行列组织结构，以平衡面积、性能和功耗。整个过程采用工厂模式实现模块化创建，支持从网表生成到物理布局的完整自动化流程。

— 2a 实例化SRAM架构

sram.py:54

```
self.s = sram(name, sram_config)
```

SRAM实例创建过程

— sram.py构造函数

— 2b 生成网表和布局

sram.py:56

```
self.s.create_netlist() if not OPTS.netlist_only: self.s.create_layout()
```

— 配置参数传播

— 2c 设置本地配置

sram_config.py:38

```
sram_config.set_local_config(self)
```

— 内存组织计算

— 2d 计算内存组织

sram_config.py:51

```
self.compute_sizes()
```

底层架构实现

- 工厂模式模块创建
- 网表生成逻辑
- 物理布局设计

3 银行级内存实现

展示单个内存银行的详细实现，包括存储阵列和外围电路的集成. [See guide](#)

🔗 AI generated guide

Motivation

现代芯片设计需要大量不同规格的SRAM存储器，但手动设计每种规格既耗时又容易出错。OpenRAM解决了这个痛点：它是一个**自动化的SRAM编译器**，能够根据用户指定的容量、字长、端口数等参数，自动生成完整的SRAM设计，包括电路网表、物理布局和时序模型。

Details

银行架构生成流程

Ask a follow-up question

3. 实例创建：将所有子模块组装成完整银行

存储阵列核心

位单元阵列[3c]是SRAM的心脏，由成千上万个存储单元组成。系统根据配置参数动态计算最优的行列组织，确保面积和性能的平衡。

物理设计自动化

最后阶段执行物理设计[3d]，包括：

- 布局：智能放置各个模块位置
- 布线：自动连接信号线和电源线

整个过程完全自动化，用户只需提供配置文件，就能获得生产就绪的SRAM设计。

SRAM架构创建

— 单银行SRAM实例化

— 3a 创建内存银行

sram_1bank.py:457

self.bank = factory.create("bank", sram_config=self.sram_config, module_name="bank")

银行模块构建过程

— 添加子模块和引脚定义

— 3b 银行模块构建

bank.py:59

self.add_modules() self.add_pins() self.create_instances()

— 存储阵列创建

— 3c 创建存储阵列

bank.py:185

self.bitcell_array = factory.create(module_type=OPTS.bitcell, cols=self.cols, rows=self.rows)

物理设计执行

— 模块布局放置

— 实例位置计算

— 信号布线连接

— 3d 物理设计执行

bank.py:64

self.place_instances() self.route_layout()

4 字符化与时序分析

展示SRAM时序和功耗特征的自动分析流程. [See guide](#)



🔗 AI generated guide

Motivation

在芯片设计中，SRAM存储器需要**精确的时序和功耗数据**才能被正确集成到更大的系统中。手工分析这些参数极其耗时且容易出错。OpenRAM的字符化系统自动生成这些关键数据，解决了从物理设计到可用内存模块之间的鸿沟。

Details

字符化流程从SRAM实例的save()方法开始[4a]，首先准备负载和转换时间表格，然后创建不同的工艺角（温度、电压、工艺变化组合）[4b]。系统会创建延迟分析对象来测量读写操作的具体时序[4c]，包括定义延迟测量的具体参数如上升/下降时间和测量触发条件[4d]。

SRAM.save() 启动输出生成

— 整个过程自动生成Liberty时序库文件，包含所有操作条件下的延迟、功耗和建立保持时间，为后续

续 4a 启动字符化分析

sram.py:191

lib(out_dir=OPTS.output_path, sram=self.s, sp_file=sp_file)

— prepare_tables() 准备参数表

— create_corners() 创建工艺角

— 4b 字符化流程初始化

lib.py:42

Ask a follow-up question



— 4d 延迟测量定义

delay.py:89

self.delay_meas.append(delay_measure("delay_lh", self.clk_frmt, targ_name, "FALL", "RISE", measure_scale...

— analysis_init() 初始化分析

— 其他输出文件生成

— sp_write() SPICE网表

— gds_write() GDS布局

— verilog_write() Verilog模型



sram_config.py

delay.py

lib.py

sram_compiler.py

sram_1bank.py

sram.py

bank.py

compiler > sram_config.py

```
1 # See LICENSE for licensing information.
2 #
3 # Copyright (c) 2016-2024 Regents of the University of California and The Board
4 # of Regents for the Oklahoma Agricultural and Mechanical College
5 # (acting for and on behalf of Oklahoma State University)
6 # All rights reserved.
7 #
8 from math import log, sqrt, ceil
9 from openram import debug
10 from openram.sram_factory import factory
11 from openram import OPTS
12
13
14 class sram_config:
15     """ This is a structure that is used to hold the SRAM configuration options. """
16
17     def __init__(self, word_size, num_words, write_size=None, num_banks=1,
18                 words_per_row=None, num_spare_rows=0, num_spare_cols=0):
19         self.word_size = word_size
20         self.num_words = num_words
21         # Don't add a write mask if it is the same size as the data word
22         self.write_size_init = write_size
23         if write_size:
24             self.write_size = write_size
25         else:
26             self.write_size = word_size
27         self.num_banks = num_banks
28         self.num_spare_rows = num_spare_rows
29         self.num_spare_cols = num_spare_cols
```

Ask a follow-up question



Ask a follow-up question

▼